1 Lesson 12-10-26

In the last lesson we introduced programmable memories, covering:

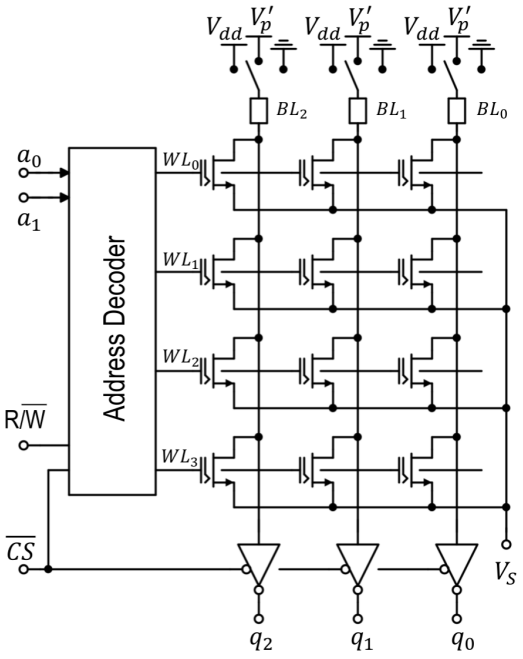
- **MPROM**s (*Mask Programmable ROMs*), programmable only once by altering the design at production time;
- **OTP-ROM**s (*One Time Programmable ROMs*), based on fixed MOSFET paths programmed only once by *blowing fuses*;
- **EPROM**s (*Erasable Programmable ROMs*), programmable electrically, and erasable only via photoelectric effects (shining a light on them). These were based on **FAMOS** (*Floating gate Avalanche MOS*) transistors;
- **EEPROM**s (*Electrically Erasable Programmable ROMs*), programmable and erasable electrically. These were based on **FLOTOX** (*FLoating gate Tunneling OXide MOS*) transistors.

1.1 Flash memory

We now cover **flash** memory storage, which seeks to lower the cost and latency problem of common EEPROM memory, while still providing a completely electric read/write interface.

1.1.1 NOR Flash

NOR Flash memory is based on FLOTOX transistors with thin layers on the source side of the channel.



All Sources are tied together and not connected to ground. All FLOTOX transistors are not programmed after fabrication, so that they behave normally and the initial content of the memory is all 1s.

NOR Flash transistors are programmed and erased in different ways:

- **Hot Electron Programming** is used for *programming*, to avoid affecting other transistors;
- **Tunneling effect** is used for erasing.

  Let's see in detail:

- To **program** the memory, the word is selected via its address and data is provided on the data bus.

  The select word line (targeting the gates) is driven at programming voltage V_p, and all other word lines are kept at ground.

  The selected bit lines (targeting the drains) are also driven at programming voltage V_p', and all other bit lines are kept at ground.

  Lastly, sources are connected to ground with $V_S = 0\,\mathrm{V}$.

  The high energy electrons which end up traveling through the channel are thus able to be injected in the gate, raising the gate threshold V_{TH} and encoding a logic 0;
- To perform a **full erase** of the memory, all word lines and bit lines are grounded.

  The source is then driven to high programming voltage.

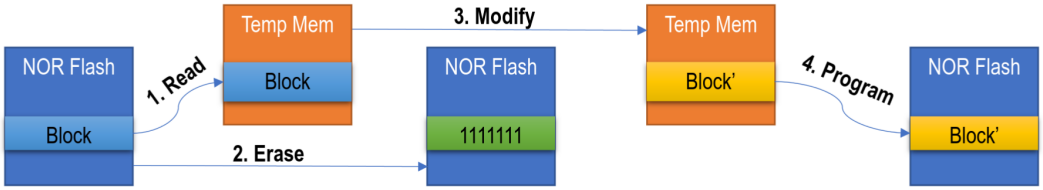
  Since the thin oxide layer is on the source side, the result is that we experience electron tunneling from gate to source, and a logic 1 is encoded in every transistor (so each memory location).
- A **block erase** is also possible by splitting the V_S line among different groups of transistors (at least as big as a single word). Usually block size ranges from a few Kbytes to hundreds of Kbytes.

  Sources of different blocks can be independently driven: the erase is therefore applied to only one block at a time.

1.1.2 NOR Flash peformance

In regards to performance, we have that the **read** operation is identical to Mask Programmable ROM, OTP-ROM, EPROM and EEPROM. The performance in **writing**, other hand, sacrifices single bit access while erasing (present on EEPROMs). When writing, an entire block is erased and rewritten, even when writing a single word.

This calls for more complex memory writing mechanisms, which need temporary buffers to store the blocks being modified. One such mechanism is detailing in the figure.

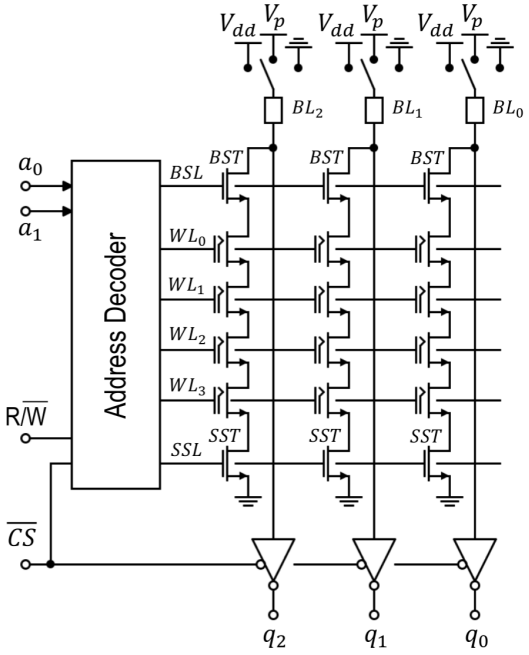


Lastly, we note that for NOR Flash memory a **CFI** (*Common Flash memory Interface*) exists, and is agreed upon and used by constructors.

- Reading is performed as in a ROM;
- Writing is based on commands: a standard set exists, and software drivers are usually used to access these kinds of memories.

1.1.3 NAND Flash

NAND Flash is the NAND version of NOR flash. It still uses FLOTOX transistors, but with thin oxide on the drain side. The difference is that tunneling is used for both *reading* and *writing* cells.



As with MPROMs (see lesson 7), NAND topology is slower (higher parasitic capacitances and resistances), but denser (lower costs and size).

To enable correct usage, access transistors are placed at the top and bottom of a series:

- At the top (drain side) we have a **BST** (*Bit Line Select Transistor*);
- At the bottom (source side) we have a **SST** (*Source Select Transistor*).

All FLOTOX transistors are not programmed after fabrication, so the initial content of the memory are all 1s.

In NAND Flash, all gate voltages are complemented compared to NOR topology. Voltage values used in NAND Flash memories are:

- V_{th} is the nominal threshold voltage of a non programmed transistor;
- V_{thprog} is the threshold voltage of a programmed transistor;
- $V_{pass} > V_{thprog}$ is a gate voltage such that all transistors are conducting, regardless of their programming state;
- V_{read} is a gate voltage such that only non programmed transistors are conducting;
- $V_{off} < V_{th}$ is a gate voltage such that all transistors are not conducting.

We move on to operations.

- First off, **reading** is non trivial. We first set all bit lines to V_{dd} (so that the pull-up transistors actually pull up).

  Then we set the selected word line to V_{read}, and all others to V_{pass}. This means that all transistors, except in the word to be read, are conducting.

  At this point:

  - If the transistor in the word is not programmed, it is conducting: the Bit Line is grounded, and the output is logic 1;
  - If the transistor in the word is programmed, it is not conducting: the series is interrupted, the Bit Line remains high, and the output is logic 0.

  Note that reading a NAND flash memory is slow, as the NAND topology has much higher delays than the NOR topology. For this reason the memory is divided in **blocks**: each block is tens or hundreds of Kbytes. Each block is divided in **pages**, where ach page is tens or hundreds of bytes. A page is a row, or a fraction of a row, of the bit cell array.

  Most NAND Flash Memories are read *page by page*, storing in a temporary SRAM buffers. Words are read sequentially within a page, and no random access to individual words is permitted. Because of this, NAND Flash Memory is not a drop-in replacement of a ROM: it cannot be used to store and execute code directly.

- **Programming** a transistor iw performed by setting the selected word line to V_p, and all the other word lines to V_{pass}.

  Then, the desired bit line is set to ground, and all the other bit lines to V_p.

  Lasly, the BSTs are enabled and the SSTs disabled, meaning the sources are floating (Which is the programming condition for FLOTOX transistors via electron tunneling).

  With this method we do program the selected transistors, and other transistors aren't affected:

  - Transistors in other words (row) have a small V_{GD};
  - Other transistors in the same word have $V_{GD} = 0\,\mathrm{V}$

- **Erasing** is done in blocks.

  BST and SST access transistors are deactivated, so that all channels are left floating.

  Then all word lines are grounded.

  The erasure tunneling happens directly via the *substrate*, which is set to progamming voltage V_p.

  This means we erase an entire *block*, or the entire memory. For this reason, blocks are in isolated substrate wells. All blocks can be erased concurrently, affecting the entire memory.

1.1.4 NAND Flash performance

We already said that NAND Flash provides very low read efficency, and as such is more suited to whole page caching, with serial access protocols.

The serial interface is similar to **SPI**, with up to 4 parallel bits (non-SPI mode).

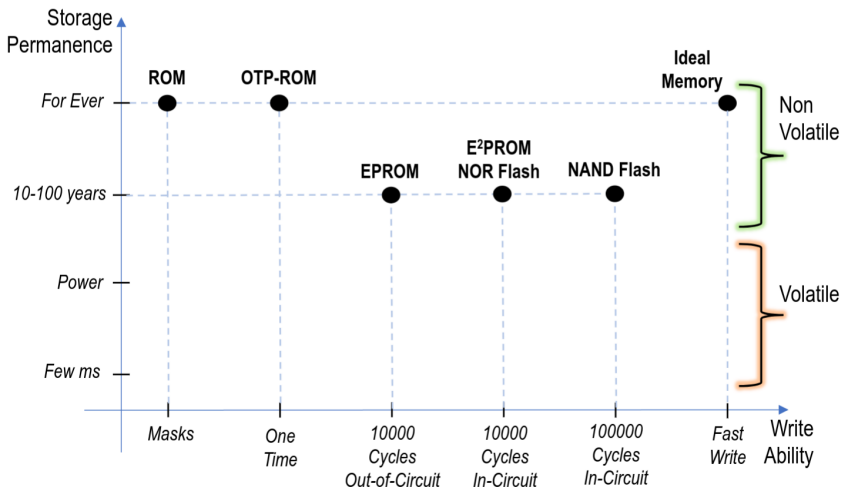
To summarize, to increase performance a page is first transferred to an internal cache (made up of SRAM). Reads and writes always occur from/to the cache, with the internal memory controller governing read/writes to and from the actual NAND memory cells.

As with NOR Flash, the protocol defines a number of commands.

Lastly, we note that we need to read – modify – rewrite, like in the NOR Flash Memory case. It's easier to write the updated block to an empty block, marking the initial one for erasing.

1.2 Summary of ROM memories

We show graph of Write Ability and Storage Permanence of ROM memories (see section 6.0.2).



We see that:

- All ROM memories have a large penalty in writing, and as such are used mostly to read data;
- Flash memories are faster (than for instance EEPROMs), but erasing and writing not at word level;
- All ROM memories are *non volatile*, but this is sometimes not necessary. In these cases we can sacrifice storage permanence by write as fast as reading, using word and bit levels (instead of programming voltages). With this method we can use standard access methods, with no need of programming and erasing.

1.3 Read Only Memories

We now move on to **RAM**s (*Read Only Memories*). RAM denotes memories that are fast to write, and random access. ROMs are also random access, but RAMs are much faster than E2PROMs and Flash memories (several order of magnitude faster especially in writing).

RAMs usually allow single bit and single word access in reading and writing, with no distinction between programming and erasing: write is identical in the two cases ($0 \rightarrow 1$ and $1 \rightarrow 0$).

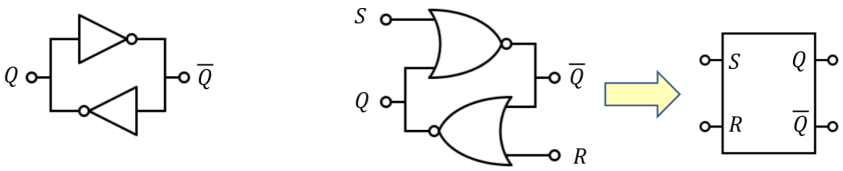
There are two main kind of RAMs:

- *Static* RAM (**SRAM**, **CAM** or *Content Addressable Memory*);
- *Dynamic* RAM (**DRAM**, **SDRAM**, **DDR**).

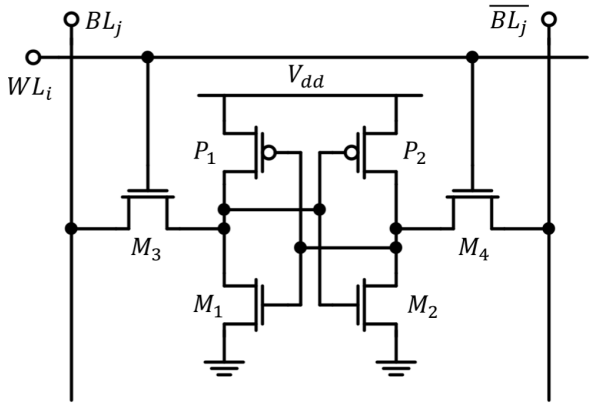
1.3.1 Static RAM

Static RAM relies on *bistable* circuits, that is the usual sequential data latches.

As we saw in 3.2, a NOT loop is a basic example of a bistable circuit, with an unstable center state.



To create an SRAM cell from an SR latch, we need a special topology.

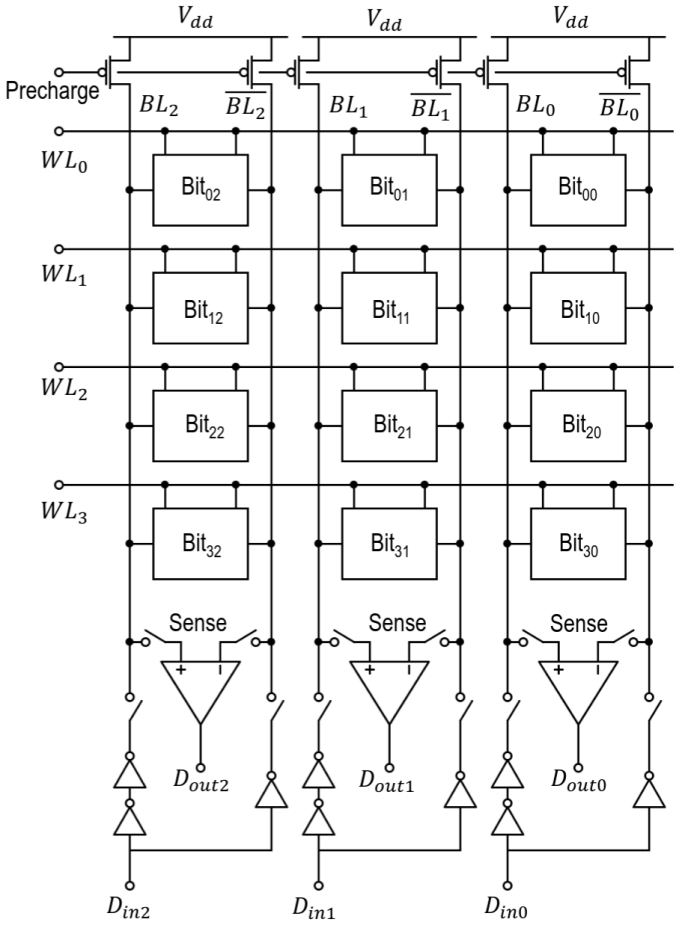


This features:

- The M_1, M_2, P_1 and P_2 transistors to realize the basic bistable cell;
- The *access* transistors M_3 and M_4, configured as pass transistors. These allow programming of the bit.

We note the two Bit Lines for each bit cell: one affirmed and one complemented. A single Word Line, on the other hand, drives the gates of both the access transistors, enabling reads/writes to the bistable cell.

Let's now look at the organization of static RAM cells to form a full SRAM:



We see that a number of components have to be studied:

- The single *bit cell* is the same as we already saw. Note that a single pair of affirmed and complemented bit lines is shared across multiple cells;
- The *precharge* circuit. This is used to precharge the bitlines to a level which won't interact with the values stored in the bit cells on reads;
- The *sense amplifier*: this is used to sense values in the bit lines and amplify them to full logic levels. All of this machinery is really needed because sharing multiple bit lines across bit cells multiplies the resistances and capacitance experienced by these;
- Lastly we need a *column decoder* and *mux* for allow I/O data lines at the bit lines.

Let's now go through *operations*.

- **Reading**. This happens in a few steps.

  1. First we charge both the bitlines to V_{dd}. Then we detach the precharge source. This means that the V_{dd} voltage is mantained through wire capacitance on the bit lines (usually in the order of picofarads);
  2. After the bitlines are left floating, the M_3 and M_4 passgates are opened. This means that one side (the high side) of the bit lines will stay high, and the other will go low. By sensing the voltage difference across them, we can recover the data which was initially stored in the bit line.

  Note that in this operation M_3 starts conducting current, thus raising the voltage at M_2's gate. To avoid bit flips, M_3 should be sized smaller than M_1 (to have a higher

R_{DSon} resistance and take more voltage in the voltage divider that is formed).

- **Writing**. This is arguably simpler.
  1. We first load the bit lines (affirmed and complemented) to respectively the affirmation and the complement of the value we want to store. This process is similar to precharging, with the difference that the voltage applied to the bit lines is kept until wire capacitance charge, and throughout the whole writing process (so the bit cells are actively driven);
  2. To actively drive the bit cells we then just raise the word line, so that the bitline values propagate to the bit cell.

  Note that for this operation to work we need to be able to force a bit flip, and thus the pass gate transistors M_3 and M_4 should be larger than the PMOS transistors P_1 and P_2 (so that their side in the voltage divider is larger).

1.3.2 SRAM performance

SRAM performance is great in both reading and writing.

- For the *read operation* we said that the low side nMOS transistors in the bistable circuit should be larger than the corresponding nMOS access transistors. This means that the ON resistance is lower for the low side nMOS transistors.

  We then have that the sense amplifier detects very small changes of the input voltages, with ΔV in the order of a few hundred millivolts.

  As such, the performance of reading is very fast, ranging from a few nanoseconds to tens of nanoseconds;
- For the *write operation* we said that the nMOS access transistors should be larger than the high side pMOS transistors in the bistable circuit. Sometimes the pMOS are implemented using resistances, or pMOS that are always conducting, designed to have a high R_{DSon}. The exact ratio of all transistors depends on technological parameters: transistors are not minimum size.

  The performance for writing is very fast, ranging from a few nanoseconds to tens of nanoseconds, which is comparable to the read operation.

  There are also *no limits* in the number of write operations (far different from most ROMs we saw), and *no distinction* between writing 0 or 1.

1.3.3 SRAM variants

A few variations on SRAM are available.

- **NVRAM** (*Non Volatile RAM*): a battery keeps power for several years. It is more expensive, larger in physical size, and heavier;
- **Multiport SRAM**: features independent ports with separate busses, independent access transistors and row and column circuits.

  This allows multiple users to read concurrently at multiple location (even the same one). They can also read a location while writing another location.

  Note that, when reading a location while writing the same location, the result of reading is not defined.

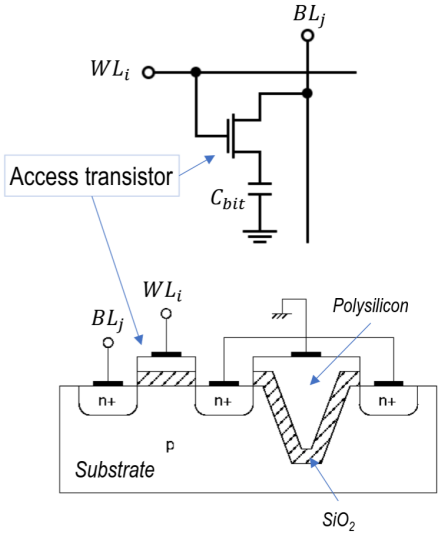
  Users can also write multiple different locations concurrently, but still note that writing the same location from different ports yields an undefined stored value.

1.4 Dynamic RAM

Our goal with dynamic RAM is to design a much smaller bit cell, to make a fast and cheap memory.

The idea is to store information as charge on a capacitor: a capacitor is a form of analog memory where if charge is present, the logic value is 1; if charge is not present, the logic value is 0.

Lastly, note that the capacity used in DRAM is actually the parasitic capacitance C_{bit} of a special transistor. A second *access* transistor allows to charge or observe the storage capacity C_{bit}.



Looking at the layout cross-section, the nMOS access transistor is a standard one. Te capacity transistor, however, is a so-called **trench capacitor**, comprised of an nMOS transistor with Source and Drain connected together. These are in turn onnected to the Source of the access transistor. The polysilicon Gate goes inside the substrate, increasing the area A of the gate. The gate is connected to ground, so that the equivalent capacitance is:

$$C_{bit} = C_{ox} \cdot A$$

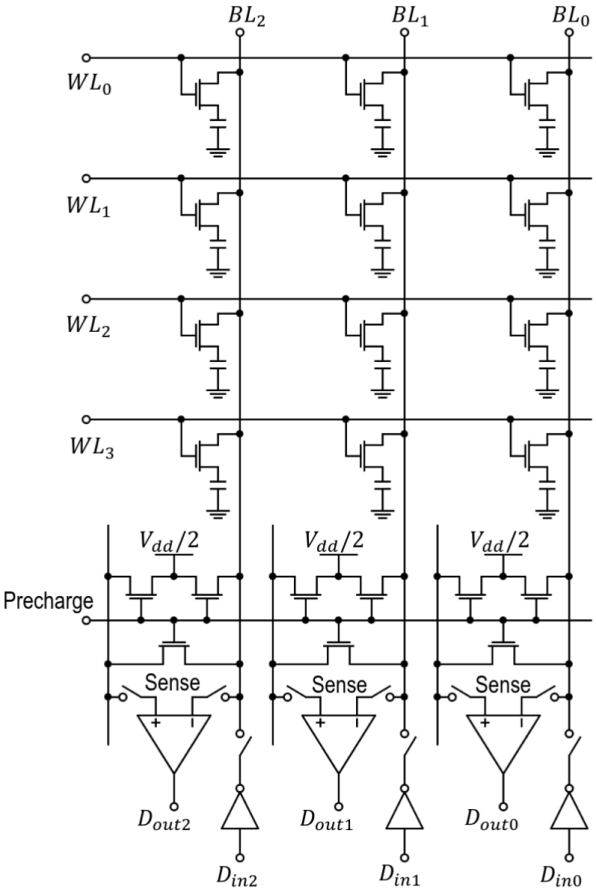
To improve the analog memory, C_{bit} should be the highest possible. Technology is defined to increase the parasitic capacitance: it cannot be embedded on the same integrated circuit as standard logic.

The DRAM bit cell is small, much smaller than SRAM. *Stacked capacitors* then allow for smaller footprint in planar technology. This allows DRAM memory to be far cheaper than SRAM.

Of course, there are problems associated with DRAM:

- Leakage currents destroy the information after some time (milliseconds), so we need to periodically refresh the data stored;
- Reading the charge on the capacitor requires to remove some charges, which need to be restored after reading to not lose data.

Let us now look at the usual DRAM topology.



We see:

- The usual matrix of bit cells, organized in word lines and bit lines. The capacity is easy to connect in this configuration as the word line just drives the access transistor gate, and the capacity drives the bit line (through the access transistor's source);
- We the have the *precharge* circuit to connect bit lines to $\frac{V_{dd}}{2}$. This is a nice unbiased voltage we can use to extract useful ΔV from the capacities;
- We then have the *sense amplifier*. It is a differential amplifier to detect small voltage differences. To have a ΔV difference in the first place, we need an additional comparison line. This is usually the $\frac{V_{dd}}{2}$ line from before;
- As with SRAM, we have the *I/O* section with inputs and outputs, switches driven by column decoder and mux.

Let's see the *operations*:

- For **reading**, we start with all wordlines inactive, and precharge the bitlines to $\frac{V_{dd}}{2}$.
  We then activate the selected wordline. Charge is thus shared between the C_{bit} capacitances and C_L, which is the cunulative capacitance of a bit line.
  When we then activate the sense amplifiers and select the desired word, the sense amplifiers detects the small change, positive or negative. This gets us the final read result.
  Note that the final voltage on the storage capacitor is close to V_{ref} (our $\frac{V_{dd}}{2}$): information is lost (we say that reading is *destructive*). To restore it we need to write after reading. This is performed internally by the memory, so it is not necessary to issue a

write command. On the other hand, rewriting is necessary also in other words of the same row (even if not read).

We can do some calculations to evaluate the charge transfer between C_{bit} and C_L in both the reading 1 and reading 0 state:

– For the 1 state, we start by computing the total charge on the C_{bit} capacity and the C_L capacity before raising the word line. Assuming that after raising the word lines these are equalized, we have to find the resulting voltage. In symbols:

$$\begin{cases} Q'_{tot} &= C_{bit}(V_{dd} - V_{th}) + C_L V_{ref} \\ Q''_{tot} &= (C_{bit} + C_L)V_X \end{cases} \implies V_X = \frac{C_{bit}(V_{dd} - V_{th}) + C_L V_{ref}}{C_{bit} + C_L}$$

At this point we compute the final ΔV as the difference between the resulting V_X and the V_{ref} the sensing amplifiers compare against:

$$\Delta V = V_X - V_{ref} = \frac{C_{bit}}{C_{bit} + C_L}(V_{dd} - V_{th} + V_{ref})$$

Lastly, note that $V_{ref} \approx \frac{V_{dd}}{2}$ as per previous discussions, so we can say;

$$\Delta V = \frac{C_{bit}}{C_{bit} + C_L}\left(\frac{V_{dd}}{2} - V_{th}\right)$$

– For the 0 state, we do the same thing in reverse. Assume the C_{bit} capacity starts low, with zero charge:

$$\begin{cases} Q'_{tot} &= 0 + C_L V_{ref} \\ Q''_{tot} &= V_X(C_{bit} + C_L) \end{cases} \implies V_X = \frac{C_L}{C_{bit} + C_L} V_{ref}$$

We compute the final ΔV the same as before:

$$\Delta V = -\frac{C_{bit}}{C_{bit} + C_L} V_{ref} = -\frac{C_{bit}}{C_{bit} + C_L}\frac{V_{dd}}{2}$$

using the same value of V_{ref} as before.

What we wanted to show is that $\Delta V_1 \approx -\Delta V_2$, so that the sensing amplifiers see basically the same voltage delta in both reading cases.

At this point we just have to note that at the end of the read operation the voltage on the storage capacitor is V_X, which is very close to V_{ref}. The original logic value is lost, and in fact all bits in the same row are lost, even if they are not part of the word that is being read. Still, the information is known at the output of the sense amplifiers! This is true for all columns, even those that were not selected: the logic value can thus be written back by driving the Bit Line with an appropriate voltage (we'll see write shortly), and the original logic value is restored on C_{bit}. This also applies to all bits in the row, as the value is taken from the sense amplifier of the corresponding column.

- For **writing**, we also start with all wordlines inactive.

  We then apply the desired to the bit lines, and precharge the bit lines of all other words to V_{ref} (as to not interfere with their values once we raise the word line).

  After this setup we raise the word line, and storage capacitors are charged or discharged to the wanted values.

  Note that this seems fast, but has the same destructive effect on bits we are not interested in as reading: we require a read (through the sense amplifiers) and a write back. This means that the performance is the same as reading.

A solution for DRAM memory is the *Comparison Line*. The sense amplifier is placed in the middle of a Bit Line: the other half of the Bit Line used for comparison.

When *precharging*, both halves of the Bit Line are precharged to V_{ref}. On word selection, only a single Word Line is active: either the upper or lower half is affected. The other half keeps the precharged value.

When *sensing*, the sense amplifier compares the two halves and gives the result. Writing back is easy as one side remembers the old value.

Of course this requires that, when *writing*, both halves are driven with the input data signal.

1.4.1 DRAM performance

We said that DRAM is slower than SRAM (by an order of magnitude) both in writing and in reading (in fact, it exhibits the same basic timing characteristics). This is because of the writeback phase required by reading and writing, both destructive operations.

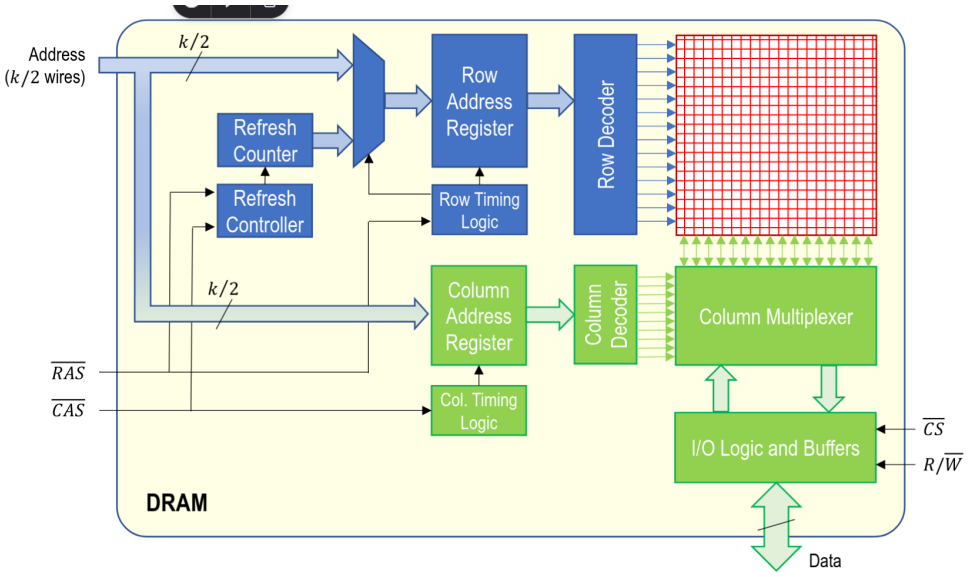
Still, DRAM has excellent write ability and very fast word and bit level writing. As with SRAM, it can be written an infinite number of times.

An important factor is the *very low storage permanence*: data is lost after a few milliseconds, and has to be **refreshed** periodically.

Assuming a DRAM has N rows and the charge on C_{bit} remains valid for at least a time T_{store}, then a new row should be refreshed every $T_{\text{refresh}} = \frac{T_{\text{store}}}{N}$.

1.4.2 DRAM interface

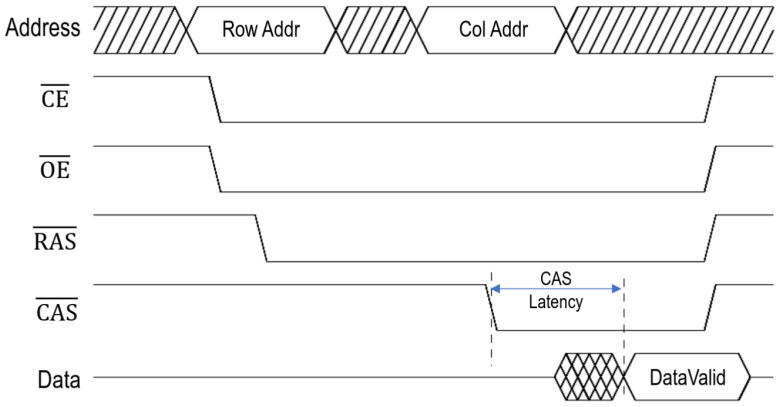
Let's look at the interface we use DRAMs with in a bit more detail.



For the overall structure we see that we multiplex two halves of the address in time, using row and column registers. This requires the additional control signals **RAS** (*Row AddreSs*) and **CAS** (*Column AddreSs*), usually active low.

As for *refreshes* (see the previous paragraph), we have that when a row is opened, all data is refreshed. Special circuits for automatic refresh are needed if refresh is automatic. Refresh can also issued by a microprocessor (CAS before RAS). Of course, as we saw, there is an *overhead* due to refresh.

Let us now look at the timings for a simple read operations, to get an idea of the typical timing of the interface.



- **Read operation** ($\overline{CE} = 0$, $\overline{WE} = 1$):
  - Present the Row Address (most significant bits) at the address pins and activate $\overline{RAS}$ (low) to latch it internally;
  - Present the Column Address (least significant bits) at the address pins and activate $\overline{CAS}$ (low) to latch it internally;
  - After a minimum delay, the output data is valid and stable.

1.4.3 DRAM variations

DRAM is relatively slow because reading data requires precharging the bit lines, selecting (opening) a row, and waiting for the sense amplifiers to detect the resulting voltage changes. Because of this, some variations exist.

- Since the row address is sufficient to open a row (or page), **Fast Page** Mode exploits the fact that all words in the same row are sensed, even those that are not explicitly read, and must also be rewritten. Once a row is open, other words within the same row can be accessed by specifying only the column address; these words need not be contiguous. To read another word in Fast Page Mode, a normal read operation is performed while keeping RAS active. CAS is then deactivated (set high), a new column address is presented at the address pins, and CAS is reactivated (set low) to latch the new column address internally. After a minimum delay, the new output data becomes valid and stable;
- *Extended Data Out* (**EDO**) Mode is similar to Fast Page Mode, but with an additional register added at the output of the memory. The Column Address can change while keeping the data of the previous operation on the DataOut Bus. This is not possible in Fast Page Mode, and the additional register implements a sort of small pipeline, increasing performance;
- *Synchronous DRAM* (**SDRAM**): all operations synchronized on the active edge of a clock. This simplifies internal structure compared to asynchronous memories. Simple circuits improve performance. These RAMs allow for contiguous addresses, column sent only once. This is useful using caches;
- *Double Data Rate* (**DDR**) is a variation on SDRAM. For contiguous data, the output is updated on both clock edges (rising and falling). For faster operation, we can use multiple clock edges, from multiple clocks out of phase.

There are different DDR DRAM variations:

- Normal applications: DDR, DDR2, DDR3, DDR4, ...
- Low Power, for mobile applications: LPDDR1, LPDDR2, ...
- High bandwidth, for graphic applications: GDDR2, GDDR3, GDDR4, GDDR5, ...
- Special architectures such as RAMBus and eDRAM.